

Návrh a implementace databázových systémů

Objektově orientované databáze

Úvod

- zatím jsme se orientovali na strukturovanou metodiku a jediný datový model – ER model
- existují další datové modely
 - objektově orientovaný (OO)
 - objektově-relační
 - XML
 - ...

Úvod

- objektově orientované databáze (OODB)
 - rozšíření nějakého OO programovacího jazyka ve smyslu perzistence objektů
 - objekty nezaniknou po skončení programu, ale trvají nezávisle na programu
- objektově-relační databáze
 - „naroubování“ některých OO principů na tradiční relační databázi
 - kompromis, vynucený neschopností OO databází zcela vytlačit stávající relační databáze

Vlastnosti relačních DBMS

- relační DBMS jsou založeny na predikátové logice 1. řádu
- silný a přitom jednoduchý dotazovací jazyk
- slabiny
 - nedokonalá reprezentace objektů reálného světa
 - normalizace vede k fragmentovaným tabulkám a nutnosti mnoha spojení
 - sémantické přetížení
 - data i vazby mezi nimi reprezentovány jedinou logickou strukturou – relací
 - nelze dobře rozlišit mezi vztahem a entitou nebo mezi více různými vztahy mezi dvěma entitami

Vlastnosti relačních DBMS

■ slabiny

- nedostatečná podpora integritních omezení
 - leckterá IO nelze v relační DB realizovat přímo a je nutné je implementovat v aplikaci – duplikace pracovního úsilí, riziko nekonzistence
- omezená množina operací
 - nelze přidávat nové typy operací nad novými datovými typy
- obtížné zpracování rekurzivních dotazů
 - v SQL neexistují prostředky pro iteraci, je nutno využít nějakou procedurální nadstavbu

Vlastnosti relačních DBMS

- slabiny

- neefektivní vnořování SQL do programovacího jazyka

- jiné datové typy, jiná vnitřní reprezentace dat, jiná logika fungování
 - => neefektivnost

- asociativní přístup

- přístup k datům je založen ryze na obsahu dat
 - navigační přístup (tedy libovolný pohyb mezi záznamy) je v relačních DB velmi omezený a neefektivní

Základní ideje objektového modelu

- silný systém datových typů
- třídy
 - zapouzdření
 - mají své instance – **objekty**
 - mohou obsahovat **metody** – procedury aplikovatelné na objekt dané třídy
 - podstatná odlišnost od „obyčejného“ strukturovaného datového typu
- každý objekt má svoji identitu (OID) nezávisle na datech, jež obsahuje
 - zjevná odlišnost od RMD
- dědičnost

Datové typy

- atomické datové typy (DT)
 - číselné typy, znakové typy, logické,
- záznamy (record structures)
 - struktura (struct), record
 - obsahují pojmenované položky (různých) DT
- kolekce (collection types)
 - množiny nebo kolekce prvků stejného DT
 - pole, seznam, množina, ...
- reference
 - odkazují na jiné objekty - pointery
- z jakéhokoli datového typu lze vytvořit nový typ pomocí **konstruktoru** (viz dále)

Třídy

- třída = typ + (volitelně) metody
- metoda
 - procedura, mající aspoň jeden parametr, a sice objekt dané třídy
- třída je abstraktní DT
 - zapouzdřuje svůj obsah, aby nebyl viditelný zvenčí
 - umožňuje manipulaci s obsahem pouze pomocí svých nabízených metod
- dědičnost
 - třídu lze odvodit od jiné třídy tak, že přebírá (a potencionálně mění) všechny její vlastnosti, může též přidávat další

Object Definition Language (ODL)

- ODL – standardizovaný jazyk
- objektový návrh
 - svět je chápán jako soustava konkrétních objektů, které mají svoji unikátní identitu
 - objekty s „podobnými vlastnostmi“ sdružujeme do **tříd**
 - objekty jsou pak instancemi nějakého abstraktního typu – třídy, podobně jako výskyty entity byly instancemi entitního typu
 - co to je ona podobnost?
 - na jedné straně konceptuální, tj. objekty jsou ve skutečném světě analogické (např. dvě různé osoby)
 - a v důsledku též obvykle datová (programová), tj. implementace objektů je stejná

Object Definition Language (ODL)

- objektový návrh
 - spočívá tedy v návrhu systému tříd, popisujících úsek reálného světa
 - není to až tak odlišné od známého ER modelování
- u tříd navrhujeme
 - atributy
 - hodnoty přiřazované objektu
 - narozdíl od RMD mohou být strukturované
 - vztahy
 - spojení objektu s jinými objekty
 - metody
 - funkce aplikovatelné na objekt

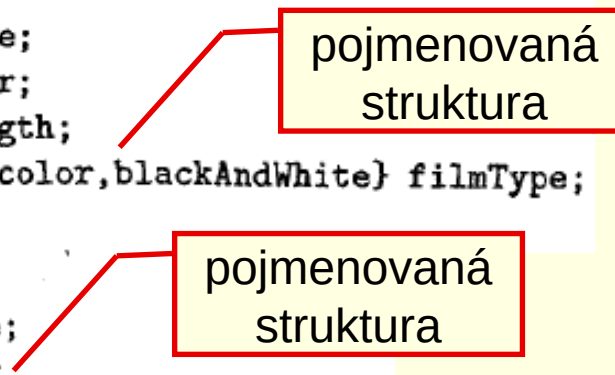
Object Definition Language (ODL)

- deklarace třídy

```
class <name> {  
    <list of properties>  
}
```

- příklady tříd pouze s atributy

```
class Movie {  
    attribute string title;  
    attribute integer year;  
    attribute integer length;  
    attribute enum Film {color,blackAndWhite} filmType;  
};  
  
class Star {  
    attribute string name;  
    attribute Struct Addr  
        {string street, string city} address;  
};
```



- objekt třídy Movie

```
("Gone With the Wind", 1939, 231, color)
```

Object Definition Language (ODL)

- pojmenované struktury z příkladu
 - nové datové typy
 - buď je lze deklarovat mimo jakoukoli třídu
 - pak jsou použitelné libovolně
 - nebo pro použití mimo třídu se na ně lze odkázat „do třídy“

```
attribute Movie::Film uses;
```

Object Definition Language (ODL)

- datové typy

- základní (basic)

- atomické – int, char, float, apod.
 - deklarované třídy – např. Movie

- strukturované – vytvořené z typu T konstruktorem

- množina: `Set<T>`

- kolekce objektů typu T bez opakování

- batoh: `Bag<T>`

- kolekce objektů typu T s možným opakováním prvků

- seznam: `List<T>`

- posloupnost objektů typu T s definovaným pořadím
 - speciálně `string = List<char>`

Object Definition Language (ODL)

■ datové typy

■ strukturované – vytvořené z typu T konstruktorem

■ pole: `Array<T, i>`

■ pole objektů typu T délky i

■ slovník: `Dictionary<T, S>`

■ množina uspořádaných dvojic

■ první položce dvojice (typu T) se říká klíč (key) a nesmí se v objektu opakovat

■ struktura: `Struct jméno {Typ1 Pol1, ...}`

■ struktura pojmenovaných položek různých datových typů

■ kolekce (collection types) jsou

■ set, bag, list, array a dictionary

Object Definition Language (ODL)

- vztahy (relationships)
 - deklarovány klíč. slovem `relationship`
 - objekt dané třídy „odkazuje“ na objekt(y) jiné třídy
 - lze použít buď atomický DT nebo nějakou kolekci
 - podle toho, zda daný vztahový atribut je či není kolekcí, se rozliší kardinalita vztahu (1:1, 1:N, N:N)
 - narozdíl od RMD, kde je propojení cizími klíči jednosměrné (n-tice druhé relace „neví“ o tom, že na ni ukazuje nějaká n-tice z první relace), zde jsou vztahy **navazovány obousměrně**
 - obě vztažené třídy mají parametr typu `relationship`, ukazující na partnera
 - deklarováno s použitím slova `inverse`

Object Definition Language (ODL)

■ vztahy

```
class Movie {
    attribute string title;
    attribute integer year;
    attribute integer length;
    attribute enum Film {color,blackAndWhite} filmType;
    relationship Set<Star> stars
        inverse Star::starredIn;
    relationship Studio ownedBy
        inverse Studio::owns;
};

class Star {
    attribute string name;
    attribute Struct Addr
        {string street, string city} address;
    relationship Set<Movie> starredIn
        inverse Movie::stars;
};

class Studio {
    attribute string name;
    attribute string address;
    relationship Set<Movie> owns
        inverse Movie::ownedBy;
};
```

odkazuje na více
hvězd (herců)

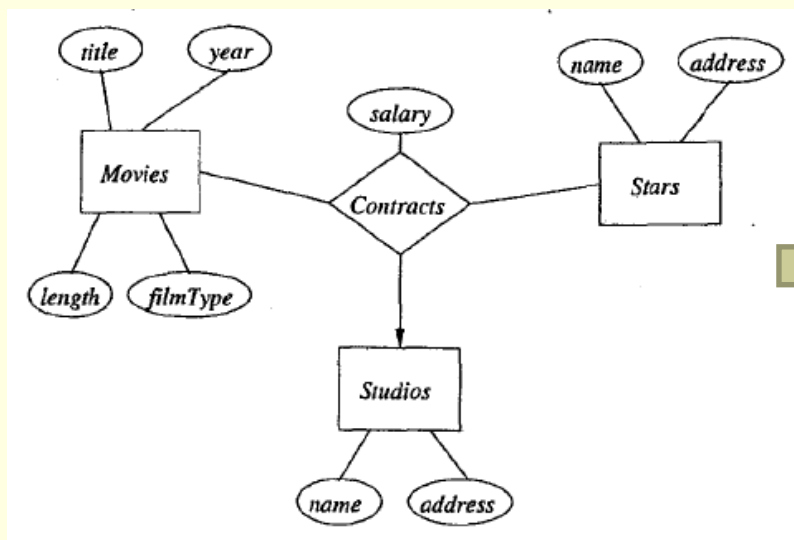
odkazuje na
jedno studio

odkazuje na více
filmů

odkazuje na více
filmů

Object Definition Language (ODL)

- vztahy (relationships)
 - opět se zde objevuje sémantický relativismus
 - vztah lze modelovat buď jako vztah (relationship) nebo jako atribut typu třída
 - ODL nemá vztahy vyšší arity (ternární apod.)
 - podobně jako RMD se řeší dekompozicí na vztahovou třídu a binární vztahy se všemi aktéry



```
class Contract {  
    attribute integer salary;  
    relationship Movie theMovie  
        inverse ... ;  
    relationship Star theStar  
        inverse ... ;  
    relationship Studio theStudio  
        inverse ... ;  
};
```

Object Definition Language (ODL)

■ metody

- deklarují se jako **signatury**, tj. deklarace sama o sobě negarantuje, že metoda bude dělat přesně to, co má

- signatura = jméno + I/O parametry + návratová hodnota

■ parametry

- vstupní (deklarované jako `in`)
 - volány hodnotou, nelze v metodě měnit

- výstupní (deklarované jako `out`)
 - volány odkazem, lze v metodě měnit

- obousměrné (deklarované jako `inout`)
 - volány odkazem, lze v metodě měnit

- metody mohou vyvolávat **výjimky** (exceptions)

Object Definition Language (ODL)

metody

```
class Movie {  
    attribute string title;  
    attribute integer year;  
    attribute integer length;  
    attribute enumeration(color,blackAndWhite) filmType;  
    relationship Set<Star> stars  
        inverse Star::starredIn;  
    relationship Studio ownedBy  
        inverse Studio::owns;  
    float lengthInHours() raises(noLengthFound);  
    void starNames(out Set<String>);  
    void otherMovies(in Star, out Set<Movie>)  
        raises(noSuchStar);  
};
```

návratová
hodnota

přidružená
výjimka

I/O parametry

Object Definition Language (ODL)

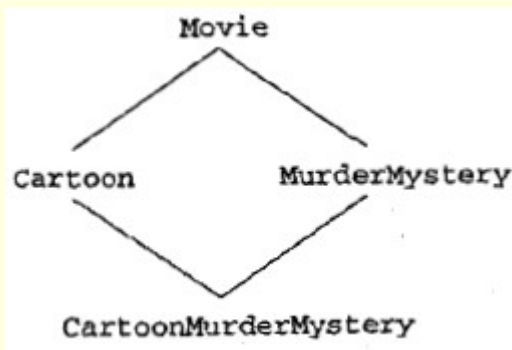
- dědičnost

- jeden rodič

```
class Cartoon extends Movie {  
    relationship Set<Star> voices;  
};
```

```
class MurderMystery extends Movie {  
    attribute string weapon;  
};
```

- více rodičů



```
class CartoonMurderMystery  
    extends MurderMystery : Cartoon;
```

- pozor na konflikty při dědění týchž vlastností!
 - přejmenovat
 - nebo specifikovat, odkud se dědí

Object Definition Language (ODL)

- vytváření instancí
 - třída je abstraktní typ
 - je třeba odlišit tuto (abstraktní) třídu od množiny jejích instancí – tzv. **extent**
 - množinu instancí je vhodné pojmenovávat např. jako jméno třídy v množném čísle

```
class Movie (extent Movies) {  
    attribute string title;  
    ...  
}
```

- třídy bez extentu – tzv. **interface (rozhraní)**
 - slouží např. jako zdroj dědičnosti

Object Definition Language (ODL)

■ klíče v ODL

- v OODB je každý objekt unikátně identifikovatelný
- tj. není třeba nic jako klíč v RMD
- přesto není špatné klíče deklarovat
 - klíč je doporučení pro DBMS, které nemusí být využito

■ jednoduchý klíč

```
class Star
    (extent Stars key name)
{
    attribute string name;
    ...
}
```

■ složený klíč

```
class Movie
    (extent Movies key (title, year))
{
    attribute string title;
    ...
}
```

Object Definition Language (ODL)

■ klíče v ODL

- lze deklarovat více kandidátních klíčů (není pak bráno jako složený klíč) – bez závorek kolem n-tice

```
class Employee
  (extent Employees key empID, ssNo)
  ...
```

- klíčem může být i metoda, pokud vrátí unikátní hodnotu pro každý objekt
- klíčem nebo součástí klíče může být i vztah (relationship) => model slabé entity

```
class Crew
  (extent Crews key (number, unitOf))
{
  attribute integer number;
  relationship Studio unitOf
    inverse Studio::crewsOf;
}
```


Transformace ODL → RMD

- ODL byl vymyšlen pro účely převodu do objektově orientovaného DBMS
- v principu ale lze provést OO návrh a pak jej nějak transformovat do RMD
 - podobné jako transformace ER → RMD
- specifika
 - ODL nemusí mít klíč
 - nutno případně zavést nějaký umělý klíč (ID)
 - ODL má metody
 - těžko řešitelné
 - podpora ve standardu SQL-99 – objektově-relační DB

Transformace ODL → RMD

■ specifika

- ODL nemusí mít pouze atomické atributy
 - řešení typu Set a Struct jsme již řešili u ER modelu (reprezentace vícehodnotových a skupinových atributů)
 - buď samostatné relace
 - nebo nenormalizované relace s redundancemi
 - Bag: u opakujících se n-tic vyznačíme počet opakování
 - List: přidáme indikátor pořadí n-tice v seznamu
 - pokud provádíme OO návrh a očekáváme převod do RMD, je nutno používat datové typy střizlivě a uvážlivě

■ vztahy

- N:N – vztahová relace
- 1:N – cizí klíč (ovšem pouze na jedné straně!)

Objektově-relační model

- z důvodu širokého nasazení relačních DB se objektovým databázím nepodařilo relační databáze vytlačit
- výrobci databází se vydali jinou cestou
 - obohatit relační databáze některými principy objektových databází
 - zpětná kompatibilita – uživatelé nemusejí OO rysy využívat, DBMS se pak chová zcela „normálně“

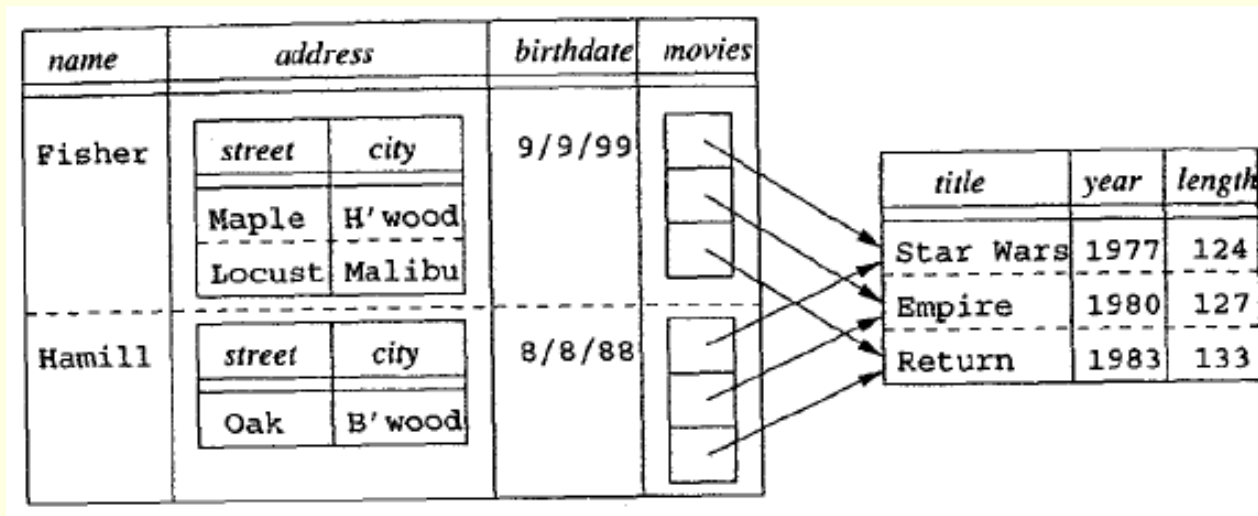
Objektově-relační model

- rozšíření relačního modelu
 - strukturované datové typy
 - tabulka již nemusí být plochá, není 1.NF
 - buňka relace může obsahovat např. kolekci struktur (set of struct), což je de facto vnořená relace

<i>name</i>	<i>address</i>	<i>birthdate</i>	<i>movies</i>																		
Fisher	<table><tr><th><i>street</i></th><th><i>city</i></th></tr><tr><td>Maple</td><td>H'wood</td></tr><tr><td>Locust</td><td>Malibu</td></tr></table>	<i>street</i>	<i>city</i>	Maple	H'wood	Locust	Malibu	9/9/99	<table><tr><th><i>title</i></th><th><i>year</i></th><th><i>length</i></th></tr><tr><td>Star Wars</td><td>1977</td><td>124</td></tr><tr><td>Empire</td><td>1980</td><td>127</td></tr><tr><td>Return</td><td>1983</td><td>133</td></tr></table>	<i>title</i>	<i>year</i>	<i>length</i>	Star Wars	1977	124	Empire	1980	127	Return	1983	133
	<i>street</i>	<i>city</i>																			
	Maple	H'wood																			
	Locust	Malibu																			
<i>title</i>	<i>year</i>	<i>length</i>																			
Star Wars	1977	124																			
Empire	1980	127																			
Return	1983	133																			
Hamill	<table><tr><th><i>street</i></th><th><i>city</i></th></tr><tr><td>Oak</td><td>B'wood</td></tr></table>	<i>street</i>	<i>city</i>	Oak	B'wood	8/8/88	<table><tr><th><i>title</i></th><th><i>year</i></th><th><i>length</i></th></tr><tr><td>Star Wars</td><td>1977</td><td>124</td></tr><tr><td>Empire</td><td>1980</td><td>127</td></tr><tr><td>Return</td><td>1983</td><td>133</td></tr></table>	<i>title</i>	<i>year</i>	<i>length</i>	Star Wars	1977	124	Empire	1980	127	Return	1983	133		
<i>street</i>	<i>city</i>																				
Oak	B'wood																				
<i>title</i>	<i>year</i>	<i>length</i>																			
Star Wars	1977	124																			
Empire	1980	127																			
Return	1983	133																			

Objektově-relační model

- rozšíření relačního modelu
 - metody
 - definice specifických operací pro typ
 - identifikátory objektů (OID) – viditelné či skryté
 - reference / pointery, které lze vložit do relace



Object Query Language (OQL)

- objektový model má svůj dotazovací jazyk
 - operuje nad daty definovanými pomocí ODL
 - jakési prolnutí SQL a objektového programovacího jazyka
 - cíl: přinést to nejlepší z SQL do OOP
 - (naproti tomu objektově-relační model, standardizovaný v SQL-99, se snaží vnést to nejlepší z OOP do SQL)
 - k datům lze přistupovat pomocí SELECT-FROM-WHERE bloku stejně dobře jako pomocí příkazů procedurálního jazyka

Object Query Language (OQL)

- tečková notace

- je-li p atribut, pak $a.p$ je hodnota atributu p v objektu a
- je-li p vztah, pak $a.p$ jsou objekty, které jsou ve vztahu p s objektem a
- je-li p metoda, pak $a.p$ aplikuje p na objekt a

Object Query Language (OQL)

■ příklad

```
class Movie
  (extent Movies key (title, year))
{
  attribute string title;
  attribute integer year;
  attribute integer length;
  attribute enum Film {color,blackAndWhite} filmType;
  relationship Set<Star> stars
    inverse Star::starredIn;
  relationship Studio ownedBy
    inverse Studio::owns;
  float lengthInHours() raises(noLengthFound);
  void starNames(out Set<String>);
  void otherMovies(in Star, out Set<Movie>)
    raises(noSuchStar);
};
```

```
class Star
  (extent Stars key name)
{
  attribute string name;
  attribute Struct Addr
    {string street, string city} address;
  relationship Set<Movie> starredIn
    inverse Movie::stars;
};
```

```
class Studio
  (extent Studios key name)
{
  attribute string name;
  attribute string address;
  relationship Set<Movie> owns
    inverse Movie::ownedBy;
};
```


Object Query Language (OQL)

- použití bloku SELECT-FROM-WHERE
 - výsledkem je *bag of objects*

```
SELECT m.year  
FROM Movies m  
WHERE m.title = "Gone With the Wind"
```

- za SELECT
 - výraz, obsahující konstanty a proměnné definované za FROM
- za FROM
 - seznam deklarací proměnných:
typ_kolekce jméno

```
SELECT s.name  
FROM Movies m, m.stars s  
WHERE m.title = "Casablanca"
```

Object Query Language (OQL)

- použití bloku SELECT-FROM-WHERE
 - lze používat DISTINCT nebo ORDER BY
 - existují též agregace či GROUP BY

```
SELECT m
FROM Movies m
WHERE m.ownedBy.name = "Disney"
ORDER BY m.length, m.title
```

- výstupem může být strukturovaný typ

```
SELECT DISTINCT Struct(star1: s1, star2: s2)
FROM Stars s1, Stars s2
WHERE s1.address = s2.address AND s1.name < s2.name
```

- lze používat poddotazy

```
SELECT DISTINCT s.name
FROM (SELECT m
      FROM Movies m
      WHERE m.ownedBy.name = "Disney") d,
      d.stars s
```

Object Query Language (OQL)

- použití procedurálního jazyka
 - SELECT-FROM-WHERE blok lze bez potíží začlenit do kódu

```
oldMovies = SELECT DISTINCT m
             FROM Movies m
             WHERE m.year < 1920;
```

```
movieList = SELECT m
             FROM Movies m
             ORDER BY m.title, m.year;
numberOfMovies = COUNT(Movies);
for(i=0; i<numberOfMovies; i++) {
    movie = movieList[i];
    cout << movie.title << " " << movie.year << " "
         << movie.length << "\n";
}
```

Výhody a nevýhody OO přístupu

■ výhody

- rozšířené modelovací možnosti
- možnost tvorby nových datových typů
- vnoření do procedurálního programovacího jazyka
- podpora dlouhotrvajících transakcí
- výrazově bohatší dotazovací jazyk
 - možnost navigačního přístupu
- pro mnoho aplikací má podstatně lepší výkon než RDBMS

Výhody a nevýhody OO přístupu

■ nevýhody

- neexistuje universální datový model
- nedostatek standardů
- nedostatek zkušeností
- složitost
- slabá podpora pohledů
- slabá podpora zabezpečení

Objektově-relační model

- typické OO vlastnosti jsou
 - rozšiřitelný systém datových typů
 - zapouzdření
 - dědičnost
 - polymorfismus
 - komplexní objekty
 - identifikace objektů
- je možné tyto vlastnosti nějakým způsobem „naroubovat“ na relační základ
 - => objektově-relační systém
 - každá implementace to ovšem řeší po svém

Objektově-relační model

- snaha o objektové rozšíření standardu
 - normy SQL-99 a SQL-2006
- zásadní koncepční problém
 - čisté objektové pojetí není pouhým rozšířením relačního pohledu, ale zcela odlišným konceptem
 - kombinace obou přístupů je tedy dost nesourodý hybrid

SQL-99

- objektové rysy SQL-99
 - UDT – user-defined types
 - podobné jako deklarace třídy v ODL
 - lze deklarovat metody

```
CREATE TYPE AddressType AS (  
    street CHAR(50),  
    city   CHAR(20)  
)  
METHOD houseNumber() RETURNS CHAR(10);  
  
CREATE TYPE StarType AS (  
    name    CHAR(30),  
    address AddressType  
);
```

```
CREATE METHOD houseNumber() RETURNS CHAR(10)  
FOR AddressType  
BEGIN  
    ...  
END;
```


SQL-99

- objektové rysy SQL-99
 - strukturovaný typ lze využít jako podklad pro tabulku

```
CREATE TABLE MovieStar OF StarType;
```

- reference

```
CREATE TYPE StarType AS (  
    name      CHAR(30),  
    address   AddressType,  
    bestMovie REF(MovieType) SCOPE Movie  
);
```

```
CREATE TYPE MovieType AS (  
    title     CHAR(30),  
    year      INTEGER,  
    inColor   BOOLEAN  
);
```

```
CREATE TABLE Movie OF MovieType (  
    REF IS movieID SYSTEM GENERATED,  
    PRIMARY KEY (title, year)  
);
```

SQL-99

- objektové rysy SQL-99
 - dekompozice vztahu N:N pomocí referencí
 - tabulku je nutno učinit referencovatelnou

```
CREATE TABLE MovieStar OF StarType (  
    REF IS starID SYSTEM GENERATED  
);
```

- vztahová tabulka

```
CREATE TABLE StarsIn (  
    star    REF(StarType) SCOPE MovieStar,  
    movie   REF(MovieType) SCOPE Movie  
);
```

- dereference
 - „sledování“ referencí – získání objektu „za ukazatelem“

```
SELECT Deref(movie)  
FROM StarsIn  
WHERE star->name = 'Mel Gibson';
```